JavaScript i interakcja użytkownika

Laboratorium 2 | Rozbudowa interfejsu sklepu

Programowanie Aplikacji WWW | Wersja 2.0 | Praca indywidualna

Prowadzący zajęcia Artur Samojluk | UWM w Olsztynie

Wstęp do ćwiczenia

JavaScript jest językiem programowania, który pozwala reagować na działania użytkownika i zmieniać zawartość strony bez jej przeładowania. HTML opisuje strukturę treści, CSS jej wygląd, a JavaScript zachowanie interfejsu. Drzewo DOM udostępnia elementy dokumentu skryptom. Zdarzenie, np. kliknięcie przycisku, uruchamia funkcję, która odczytuje dane, wykonuje obliczenia i aktualizuje widok.

Cel ćwiczenia: zrozumienie procesu tworzenia interakcji z perspektywy projektanta. Nauczysz się łączyć dane, logikę programu i interfejs oraz rozpoznawać błędy. Samodzielne pisanie kodu przygotowuje do zaawansowanego programowania z użyciem technologii GPT, w którym trzeba umieć ocenić i poprawić wygenerowane rozwiązania.

Rozbuduj sklep z laboratorium 1 o zmianę motywu, zegar, wyszukiwanie produktów i lokalny podgląd zapytania. Zachowaj semantyczny HTML, responsywność i obsługę klawiatury.

Zasady samodzielnej pracy

Pisz kod ręcznie. Możesz korzystać z dokumentacji, DevTools, podpowiedzi nazw i formatowania. Wyłącz generowanie kodu przez GPT, Copilot i podobne narzędzia. Nie używaj gotowych skryptów ani frameworków. Przed uruchomieniem przewidź wynik; w przypadku błędu najpierw samodzielnie ustal jego przyczynę.

1 Przygotuj strukturę i podłącz skrypt

Skopiuj działający projekt do katalogu `lab2/` w tym samym repozytorium. Zachowaj `lab1/`. Dodaj `js/main.js` i dołącz go w `head` każdej strony:

```
<script src="js/main.js" defer></script>
```

Dla zewnętrznego skryptu klasycznego `defer` pozwala pobierać plik podczas analizowania HTML i wykonać go po zbudowaniu DOM. Uruchamiaj projekt przez lokalny serwer HTTP. W DevTools sprawdź pobranie pliku JS oraz konsolę.

Używaj `const` i `let`, funkcji oraz `addEventListener`. Zdarzenia zapisuj w JS, bez atrybutów `onclick` i `onload`. Podziel inicjalizację na funkcje dla poszczególnych elementów; brak elementu na danej podstronie nie może przerywać działania całego skryptu.

2 Zmień motyw i zapamiętaj wybór

Na każdej stronie dodaj przycisk „Tryb ciemny” z `type="button"`. Przełączaj atrybut `data-theme` na elemencie `html` między `light` i `dark`. Kolory tła, tekstu, kart i pól formularza określ w CSS przez zmienne. W obu motywach zachowaj czytelność i widoczny fokus.

Aktualizuj `aria-pressed` przycisku zgodnie ze stanem motywu. Zapisz wybór w `localStorage` i odtwórz go po odświeżeniu oraz przejściu na inną podstronę. Przy braku poprawnej zapisanej wartości zastosuj motyw jasny.

Zegar katalog i formularz

3 Wyświetl datę i zegar

Na `index.html` pokaż bieżącą datę z pełnym rokiem oraz czas w formacie 24-godzinnym z sekundami. Użyj `Date` i `Intl.DateTimeFormat` z lokalizacją `pl-PL`. Napisz funkcję aktualizującą widok i wywołaj ją od razu po uruchomieniu strony.

Dodaj przyciski „Start zegara” i „Stop zegara”. Odświeżaj widok za pomocą `setInterval` co 1000 ms; przekazuj funkcję, nie tekst z kodem. Przechowuj identyfikator interwału i zatrzymuj go przez `clearInterval`. Wielokrotny Start nie może tworzyć kolejnych interwałów. Po wznowieniu odczytaj aktualny czas, zamiast zliczać upływające sekundy.

4 Filtruj produkty bez przeładowania

Nad katalogiem dodaj pole `input` typu `search` z widoczną etykietą oraz licznik wyników. Obsłuż zdarzenie `input`: pobierz zapytanie i usuń skrajne spacje przez `trim`. Sprawdź, czy nazwa produktu zawiera wpisany fragment, bez rozróżniania wielkości liter.

Przejdź po kartach, np. przez `querySelectorAll` i `forEach`. Użyj właściwości `hidden` do ukrywania niedopasowanych kart. Puste zapytanie przywraca wszystkie produkty. Przy braku wyników pokaż czytelny komunikat. Licznik i komunikat umieść w obszarze `aria-live="polite"`. Nie usuwaj kart z dokumentu; sprawdź, czy CSS nie nadpisuje ich ukrycia.

5 Oblicz wartość i pokaż podsumowanie

Na `kontakt.html` rozbuduj formularz z laboratorium 1. Przygotuj tablicę trzech obiektów produktów z polami `id`, `nazwa` i `cenaGrosze`. Identyfikatory powiąż z wartościami opcji wyboru produktu. Nazwy i ceny mają być zgodne z katalogiem.

- Po zmianie produktu lub liczby sztuk aktualizuj wartość zapytania. Zamień ilość na liczbę i sprawdź, czy jest całkowita oraz mieści się w zakresie 1–5. Dla pustych lub błędnych danych wyświetl instrukcję, a nie wynik liczbowy.
- Obliczenia wykonuj w całkowitych groszach. Napisz funkcję przyjmującą cenę i ilość, zwracającą iloczyn bez odwołań do DOM. Wynik wyświetl w PLN przez `Intl.NumberFormat` dla `pl-PL`, po przeliczeniu groszy na złote.
- Obsłuż `submit` i użyj `preventDefault`, aby pokazać lokalny podgląd bez przeładowania. Zachowaj ograniczenia HTML. Dodatkowo odrzuć wiadomość, która po `trim` ma mniej niż 10 znaków; pokaż błąd przy polu i przenieś na nie fokus. Po poprawieniu usuń komunikat.
- Dla poprawnych danych pokaż produkt, ilość, wartość i treść wiadomości. Wstawiaj dane użytkownika przez `textContent`. Podsumowanie aktualizuj przy każdym poprawnym wysłaniu; po zmianie danych usuń poprzednie potwierdzenie, aby nie pokazywać nieaktualnej wersji.

Zmień opis formularza na „Podgląd lokalny. Formularz nie wysyła wiadomości”. Ceny i walidacja w przeglądarce są demonstracją; w docelowym sklepie serwer musi je niezależnie sprawdzić.

Testy i samodzielne wyjaśnienie kodu

6 Sprawdź zachowanie i znajdź przyczynę błędu

W `TESTY.md` podaj przeglądarkę i wersję oraz oczekiwany i zaobserwowany wynik każdej poniższej próby. Jeśli wykryjesz błąd, zapisz jego przyczynę i wprowadzoną poprawkę.

Próba	Oczekiwany wynik
Motyw i nawigacja	Zmiana motywu utrzymuje się po odświeżeniu i zmianie podstrony; aria-pressed odpowiada stanowi.
Start trzy razy potem Stop i Start	Działa jeden interwał; Stop zatrzymuje aktualizację, a wznowienie pokazuje aktualny czas.
Fragment nazwy ze spacjami i inną wielkością liter	Pasujące karty pozostają widoczne; licznik zgadza się z widokiem.
Brak dopasowania i wyczyszczenie pola	Pojawia się komunikat o braku wyników; wyczyszczenie przywraca trzy karty.
Cena 1999 gr oraz ilość 3	Wynik wynosi 59,97 zł; sprawdź samą funkcję obliczeniową również w konsoli.
Ilość pusta 0 6 oraz 1,5	Wycena nie pokazuje błędnej wartości; formularz nie przechodzi do poprawnego podsumowania.
Dziesięć spacji oraz tekst Test	Same spacje są odrzucone; poprawna wiadomość ze znacznikami jest pokazana jako dosłowny tekst.
Klawiatura widoki i wyłączony JavaScript	Kontrolki działają klawiaturą przy 375 i 1280 px; bez JS katalog, treść i menu pozostają dostępne.

W DevTools ustaw breakpoint w funkcji obsługującej zdarzenie i pokaż wartości zmiennych. W zegarze sprawdź liczbę wywołań po wielokrotnym Start. W obsłudze zdarzenia celowo wywołaj funkcję o błędnej nazwie, odczytaj komunikat i numer linii, a następnie popraw kod. Końcowa konsola ma być wolna od błędów.

Oddanie i sprawdzenie samodzielności

Do rozpoczęcia kolejnych zajęć umieść ukończony katalog `lab2/` w swoim publicznym repozytorium GitHub i przekaż prowadzącemu link. Repozytorium jest źródłem prawdy o projekcie i terminowości pracy; liczy się wersja dostępna przed rozpoczęciem kolejnych zajęć. Zachowaj historię commitów dokumentującą postęp.

Dołącz HTML, CSS, JS, grafiki, `TESTY.md` i `README.md` z instrukcją uruchomienia. W README opisz przepływ: zdarzenie, dane wejściowe, obliczenie, zmiana DOM. Sprawdź uruchomienie po pobraniu repozytorium.

Przy oddaniu wykonaj samodzielnie zmianę wskazaną przez prowadzącego, np. dodanie wyszukiwania po opisie lub zmianę maksymalnej ilości. Przed edycją wskaż odpowiedni kod i przewidź efekt. Wyjaśnij działanie `defer`, różnicę między tekstem a liczbą oraz sposób uniknięcia kilku interwałów.

Dokumentacja do pracy

Skrypty i defer | Zdarzenia | localStorage

Timery | Formatowanie dat | Formatowanie cen | textContent